In422 Real-Time Information System
Project Report

Written by:

Lucas LAVIGNE

IPSA - Institut Polytechnique des Sciences Avancées

School Year 2025-2026

Contents

1 Introduction 2
2 C Code and Execution Time Statistics 2
3 Scheduling Analysis 2
3.1 Schedulability 2
3.2 Determining the Best Schedule 3
4 Conclusion 4

1 Introduction

This report presents the project for the final assignment of the real-time information system course. The objective is to study the behavior of a simple C code and find the best schedule to integrate this task into a task set.

2 C Code and Execution Time Statistics

The C code performs a simple multiplication of two large random numbers, ranging between 1 and 1,000,000. A Python script compiles the C code, executes it 1,000 times using the `subprocess` module, and calculates the execution time.

The statistics of the execution time are as follows:

- Minimum execution time: 5.95 ms
- Worst-Case Execution Time (WCET): 86.40 ms
- Q1 execution time: 6.19 ms
- Median (Q2) execution time: 6.52 ms
- Q3 execution time: 6.92 ms

3 Scheduling Analysis

3.1 Schedulability

To determine if the task set is schedulable, we use the utilization-based test for Rate Monotonic Scheduling (RMS). The utilization (U) of a task set is calculated as:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i}$$

Where C_i is the worst-case execution time (WCET), T_i is the period of the task, and n is the total number of tasks.

In our case, using the measured WCET of 86.40 ms for our C code (assigned to τ_1), we consider the following task table:

Task	**WCET (C_i in ms)**	**Period (T_i in ms)**
τ_1	86.40	10
τ_2	3	10
τ_3	2	20
τ_4	2	20
τ_5	2	40
τ_6	2	40
τ_7	3	80

The utilization of the task set is calculated as follows:

$$U = \frac{86.40}{10} + \frac{3}{10} + \frac{2}{20} + \frac{2}{20} + \frac{2}{40} + \frac{2}{40} + \frac{3}{80}$$

$$U = 8.64 + 0.3 + 0.1 + 0.1 + 0.05 + 0.05 + 0.0375 = 9.2775$$

Since the utilization is significantly greater than 1, the task set is not schedulable under RMS. Therefore, for the remainder of this study, we will assume an optimized execution time of 3 ms for τ_1.

The new utilization factor is calculated as follows:

$$U = \frac{3}{10} + \frac{3}{10} + \frac{2}{20} + \frac{2}{20} + \frac{2}{40} + \frac{2}{40} + \frac{3}{80}$$

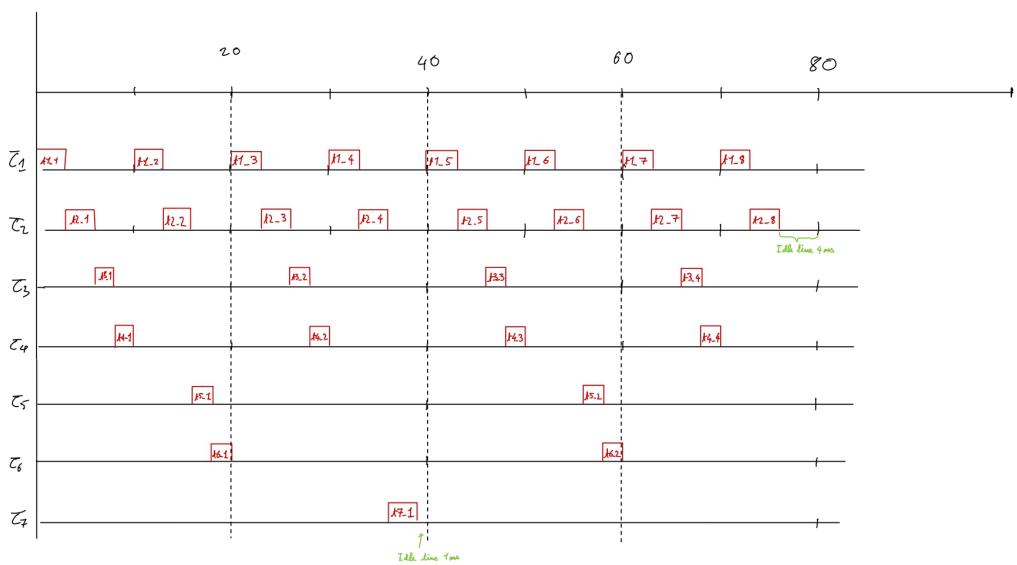
$$U = 0.3 + 0.3 + 0.1 + 0.1 + 0.05 + 0.05 + 0.0375 = 0.9375$$

Since the new utilization is less than 1, the task set is now schedulable under RMS.

3.2 Determining the Best Schedule

The hyperperiod of the task set is calculated as the least common multiple (LCM) of the task periods. The periods are 10, 20, 40, and 80 ms. The LCM of these periods is 80 ms. Therefore, the hyperperiod of the task set is 80 ms.

First, we list all the possible schedules for the task set. Then, we calculate the response time for each task in each schedule to determine if the tasks meet their deadlines. After analyzing the schedules, we identify the optimal one that allows all tasks to meet their deadlines while minimizing the response time. In this case, the best schedule is represented by the following graph:



We obtain the following results:

Because the hyperperiod, C_i, and T_i are fixed parameters defined entirely by the task set, both expected idle time (E_{total}) and actual idle time (I_{total}) are fundamental constants of the system. A scheduling algorithm can neither create nor destroy computational work.

Therefore, minimizing the total waiting time across all jobs does not maximize the total processor idle time. Instead, an optimized schedule simply alters the distribution of the idle time. By minimizing delays and executing jobs as efficiently as possible, the algorithm consolidates the idle periods into larger, contiguous blocks (typically toward the end of the scheduling cycles), rather than fragmenting them into small, inefficient gaps between task executions.

4 Conclusion

In conclusion, this project provided a comprehensive analysis of a real-time system's behavior, from execution time profiling to task scheduling. Initially, we observed that the worst-case execution time (WCET) of the C program was too high (86.40 ms) to be integrated into the given task set under Rate Monotonic Scheduling (RMS), as the utilization factor exceeded 1. By assuming a bounded execution time of 3 ms, we successfully achieved a schedulable system with a valid utilization factor of 0.9375.

Furthermore, the scheduling analysis over the 80 ms hyperperiod highlighted a fundamental principle of real-time systems: while an optimized scheduling algorithm can minimize total waiting times and ensure all deadlines are met, it cannot alter the total processor idle time. Instead, an efficient schedule merely redistributes this idle time, consolidating it into larger blocks rather than fragmented delays. This project successfully demonstrated the importance of accurate execution time profiling and the practical implications of scheduling strategies in real-time environments.